
EasyBalance: Cross-Layer Load Balancing in Distributed MoE Inference

Yize Wu^{1,2} Ke Gao¹ Ling Li^{1,2} Yanjun Wu¹

Abstract

Load Balancing has emerged as a critical problem in expert-parallel distributed inference of Mixture-of-Experts (MoE) models. As routing distributions are typically skewed across experts, devices hosting lighter-loaded experts must idle to wait for the heaviest during expert computing, leading to inefficiency. Existing load-balancing approaches primarily rely on expert replication or migration within each layer, which introduce additional overhead and limit their flexibility and scalability. To address this problem, we propose EasyBalance, a **cross-layer** load balancing strategy that requires no modifications to the expert-device mapping, enabling instant adaptability and incurring essentially no additional overhead. Our key insights are that (1) experts of other layers can be viewed as naturally redundant for the current layer, and (2) cross-layer MoE workloads can be jointly executed to mitigate their individual imbalance. Based on these observations, EasyBalance greedily schedules a subset of cross-layer workloads to run at each MoE step and defers the remaining workloads for future balancing opportunities, effectively leveraging cross-layer imbalance mitigation. Extensive experiments across models, tasks, and configurations demonstrate that EasyBalance consistently accelerates distributed MoE inference, reducing GPU idling by mostly over 40%. Code is available at <https://github.com/yize-wu/EasyInfra>.

1. Introduction

The rapidly growing parameter sizes of transformer-based models (Vaswani et al., 2017) present substantial compu-

tational challenges (Hoffmann et al., 2022). Mixture-of-Expert (MoE) architecture (Jacobs et al., 1991) offers a solution by replacing dense feed-forward layers with multiple smaller expert networks, where each token activates only a sparse subset of them. This sparse activation mechanism significantly reduces per-token computation cost (Ferdus et al., 2022), while enabling models with substantially larger overall parameter counts.

In distributed MoE inference, experts are typically dispatched across multiple devices to parallelize their computation workloads, known as expert parallelism. In this setting, when tokens are routed to experts residing on remote devices, the tensors of their hidden representations are first dispatched to the corresponding devices via an all-to-all communication. Then, after expert computation, a second all-to-all gathers the results for aggregation (Huang et al., 2024; Hwang et al., 2023).

While expert parallelism can substantially accelerate MoE inference, load imbalance has emerged as a critical performance bottleneck (Liu et al., 2024). Despite the use of load-balancing auxiliary loss during MoE training, inference-time token routing often remains uneven across experts, leading to skewed computational workloads among devices. All the devices must wait for the most heavily loaded one to finish its computation, as the operations of result gathering and aggregation require synchronization. This results in severe resource under-utilization and system inefficiency (Liu et al., 2025b).

Existing methods for mitigating load balancing primarily rely on expert replication or migration within each expert layer (Li et al., 2023; DeepSeek AI, 2025; Doucet et al., 2025). By modifying the expert-device mapping of MoE models in the system, these methods attempt to redistribute workloads more evenly across devices. Despite their effectiveness, such approaches suffer from two inherent limitations. First, they lack flexibility when serving newly assigned tasks, as expert routing patterns can vary substantially across inputs and applications (see Figure 7). Second, expert replication incurs considerable memory overhead, while expert migration introduces additional communication costs, both of which pose limitation on scalability, which is a critical concern as load imbalance becomes increasingly severe with larger-scale deployments (as demonstrated in

¹Intelligent Software Research Center, Institute of Software, CAS, Beijing, China ²University of Chinese Academy of Sciences, Beijing, China. Correspondence to: Yanjun Wu <yanjun@iscas.ac.cn>.

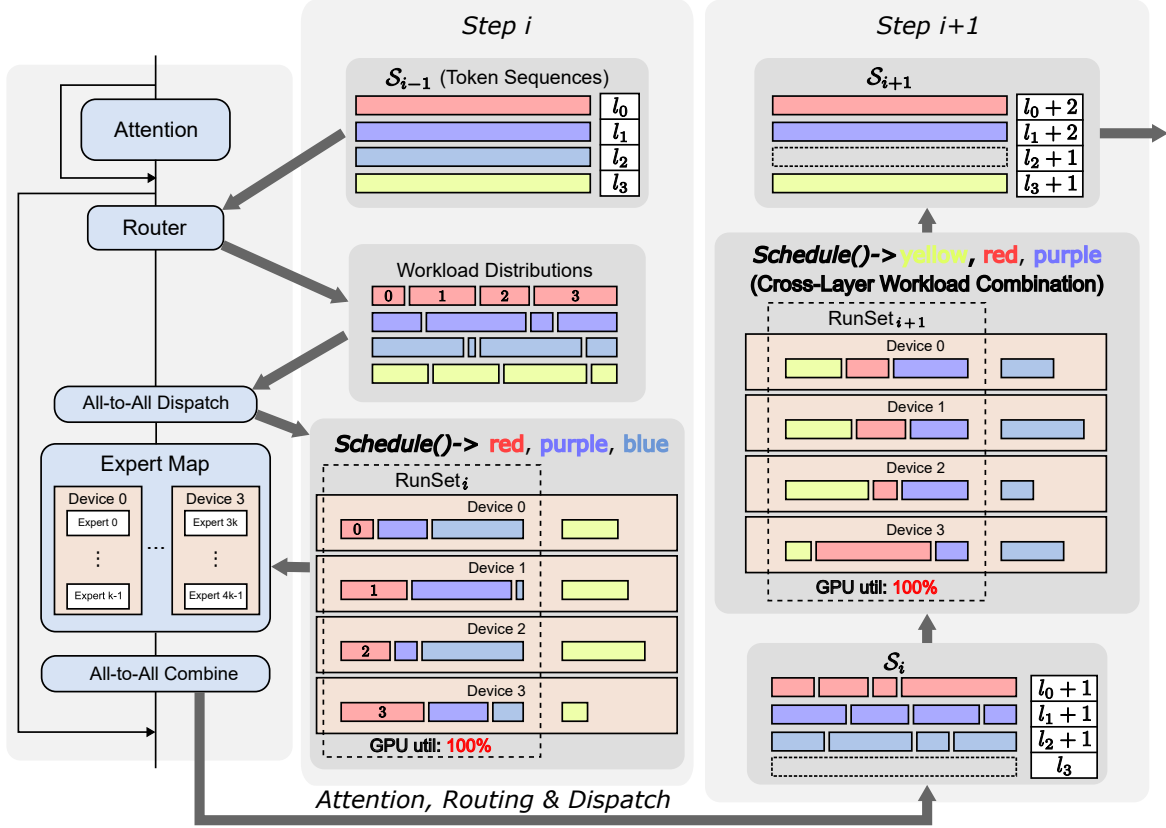


Figure 1. Demonstration of EasyBalance. MoE workloads of micro-batches (indicated by colors) can be selectively scheduled and jointly executed while residing at different layers, achieving improved GPU utilization. l_j indicates the current layer of micro-batch j .

Table 4).

To overcome these limitations, we propose EasyBalance, a novel cross-layer load balancing strategy. Unlike prior approaches, EasyBalance requires no changes to the expert-device mapping throughout the inference process, thereby providing instant task adaptability with essentially no additional memory or communication overhead. EasyBalance is motivated by two key observations:

Cross-Layer Expert Redundancy. Expert replication (DeepSeek AI, 2025) creates load-balancing opportunities by introducing redundant experts, but at the cost of increased memory consumption. However, from a novel cross-layer perspective, we observe that redundancy already exists in an inference system: for a given expert layer, experts of other layers can be regarded as naturally “redundant”, as they already reside in device memory (just like the replication). These experts can therefore be leveraged to balance workloads without introducing any additional overhead, yet this potential remains largely unexplored.

Cross-Layer Workload Combination. While model inference must follow a strict layer-wise sequential order, different micro-batches can progress through multiple layers

simultaneously, which allows MoE workloads from multiple layers to be jointly executed in one step. As shown in Figure 1, the sequences can run on different MoE layers (l_j and l_k can be different for $j \neq k$, $j, k \in \{0, 1, 2, 3\}$) with their workloads combined. Crucially, workload combination is worst-case performance-safe and mostly beneficial: (1) for two workload distributions $w_1, w_2 \in \mathbb{Z}^D$, the combined workload will be $w_1 + w_2$. Since MoE computation is bottlenecked by the heaviest rank, (i.e., $\max(w)$), the actual system workloads would satisfy the worst-case safe condition of $\max(w_1 + w_2) \leq \max(w_1) + \max(w_2)$. This property extends to multiple workloads as well; (2) the probability of encountering the worst case is low, as it only occurs when multiple workloads skew on the same device, and this probability even decreases as the number of devices scales up. Therefore, cross-layer workload combination is typically beneficial for modern deep models, particularly in large-scale distributed settings (see detailed explanation in Section 4.1, and empirical evidences in Table 2).

Based on these observations, EasyBalance performs cross-layer scheduling over micro-batches to mitigate load imbalance within the whole inference process. At each MoE computation step, it selectively schedules a subset of work-

loads for execution and defers others to wait for potential future balancing opportunities, to leverage the aforementioned property of cross-layer imbalance mitigation. Various heuristic scheduling algorithms have all been proven effective in practice, demonstrating the robustness of our method. Experimental results across models and tasks demonstrate that EasyBalance can consistently accelerate the inference process by mitigating load imbalance, with GPU under-utilization reduced by mostly over 40%.

2. Preliminary

2.1. Mixture-of-Expert Architecture

A transformer layer typically consists of two main modules: self-attention and multi-layer perceptron (MLP). Given the input hidden state h_l of layer l , the output of each module is aggregated with residual connections (He et al., 2016) as:

$$h'_l = h_l + \text{Attention}(h_l)$$

$$h_{l+1} = h'_l + \text{MLP}(h'_l)$$

The Mixture-of-Expert (MoE) architecture replaces the dense MLP with a set of experts, which are sparsely activated through a routing mechanism. For each token, the router selects the top- k experts according to the gating function G :

$$W_l, I_l = \text{topk}(G(h'_l)).$$

Each selected expert processes the token independently, and their outputs are aggregated using a weighted sum:

$$\text{MLP}(h'_l) = \sum_{i \in I_l} W_{l,i} * \text{expert}_i(h'_l), \quad (1)$$

where $W_{l,i}$ denotes the routing weight for expert i . The aggregated result is then passed to the next layer as the output of the MoE module.

2.2. Expert Parallelism

In distributed MoE inference, expert parallelism (EP) is commonly adopted to dispatch expert computation across multiple devices. Each expert is placed onto one or more devices according to an expert-device mapping, and each device hosts a subset of experts and processes tokens routed to them.

Under EP, tokens may be assigned to experts that reside on remote devices. To solve this, an all-to-all communication is required to dispatch token representations to the corresponding devices, and after MoE computation the expert outputs will be communicated back to the original devices via another all-to-all, where the weighted aggregation (Equation (1)) is performed.

Model	End-to-end Latency	
	w/o micro-batch	w/ micro-batch (4 splits)
Q3-30B	10.15	8.60
M-16B	5.06	4.40

Table 1. End-to-end latency(s) of Qwen3-30B (Q3-30B) and Moonlight-16B (M-16B) on 2wikimqa and EP=8. The batch size and sequence length are 128 and 4K.

2.3. Load Balancing

Although MoE training typically incorporates an auxiliary load-balancing loss to encourage balanced token routing, the routing distribution at inference time still often remains skewed, leading to uneven workload distribution across devices under expert parallelism. As the gathering communication and weighted aggregation are both synchronous operations, devices with lighter workloads must idle while waiting for the most heavily-loaded one to finish, introducing hardware under-utilization and therefore suboptimal performances.

Formally, denote the MoE computation workload at layer l across D devices as $\mathbf{w}^{(l)} = (w_1^{(l)}, \dots, w_D^{(l)})$. Distributed MoE computation is bottlenecked by the slowest device, so the effective workload will be:

$$\hat{w}^{(l)} = \max(\mathbf{w}^{(l)}), \quad (2)$$

which is directly proportional to execution latency. Moreover, the overall device utilization across all the layers is defined as:

$$u = \frac{\sum_{l=1}^{l_{max}} \sum_{i=1}^D w_i^{(l)}}{\sum_{l=1}^{l_{max}} \hat{w}^{(l)} \times D}, \quad (3)$$

which quantifies hardware efficiency by measuring the ratio of used resources to total available capacity.

2.4. Micro-Batching

Micro-batching is a widely used technique in large-scale inference scenarios. It splits a large batch of input sequences into multiple micro-batches and pipelines their execution (Kwon et al., 2023; SGLang Team, 2024). The benefits include computation-communication overlapping (Wang et al., 2025), efficient memory footprint, KV cache transfer in prefilling-decoding disaggregation (Yan et al., 2025), etc.. For example, empirical results in Table 1 demonstrate that micro-batching can significantly reduce end-to-end inference latency by hiding the communication overhead behind other batches' computation. EasyBalance builds upon micro-batching to further enable cross-layer workload balancing during MoE computation.

Task	N	
	2	3
2wikimqa	2/47	0/46
trec	4/47	0/46
repobench-p	10/47	1/46
all	77/611=0.126	4/598=0.006

Table 2. Numbers of N -consecutive layers that have identical workload distribution skew. The tested model is Qwen3-30B (48 MoE layers), with 32 batches and 4K sequence length. “all” stands for the statistics from all 13 representative tasks in LongBench.

3. Problem

Existing approaches address load imbalance through expert replication and/or migration within each MoE layer, to achieve more balanced workload redistribution across devices. Specifically, experts replication allow tokens routed to “hot” experts to be selectively dispatched across multiple copies, while expert migration alleviates device hotspots by relocating hot experts to other less-loaded GPUs. These techniques typically require modifying the expert-device mapping based on routing-distribution statistics.

Despite their effectiveness, current solutions suffer from two inherent limitations that restrict their applicability in broader inference scenarios:

Flexibility. MoE routing patterns are highly task-dependent (as shown in Figure 7), and an effective expert mapping for a specific task may become suboptimal—or even harmful—for inputs from other tasks. Therefore, the reliance on task- or data-specific information fundamentally limits the flexibility.

Scalability. Updating the expert mapping incurs non-negligible overheads. Expert replication introduces substantial memory consumption, while expert migration incurs additional communication and synchronization costs. These overheads scale positively with both model sizes and distribution scales, limiting the application in large-scale settings.

The limitations motivate the need for a fundamentally different load-balancing method—one that does not require modification of the expert mapping and remains both flexible and scalable across diverse inference scenarios.

4. Method

To address the aforementioned limitations, we propose EasyBalance, a cross-layer load balancing strategy for distributed MoE inference acceleration. EasyBalance requires no modification to the expert mapping, thereby offering instant flexibility and superior scalability with essentially no additional

Algorithm 1 EasyBalance

Input: Micro-batches of tokens $T_0, \dots, T_{N-1}$, minimum combination size m .
Initialize the variables.
 $l_j \leftarrow 0$ for $j \in \{0, \dots, N-1\}$
 $\mathcal{S}_0 \leftarrow \{T_0, \dots, T_{N-1}\}$
 $\text{RunSet}_0 \leftarrow \{T_0, \dots, T_{N-1}\}$
 $i \leftarrow 0$
The loop continues until all batches finish.
while $|\mathcal{S}_i| \neq 0$ **do**
 # Increase the step.
 $i \leftarrow i + 1$
 # Run other operations of last-step executed batches.
 $\text{Attn_Route_Dispatch}(\text{RunSet}_{i-1})$
 # Choose a m -minimum combination.
 $\text{RunSet}_i \leftarrow \text{Schedule}(\mathcal{S}_{i-1}, m)$
 # Execute the scheduled workload.
 $\text{MoECompute_Combine}(\text{RunSet}_i)$
 # Increase l_j of scheduled batches.
 for $T_j \in \text{RunSet}_i$ **do**
 $l_j \leftarrow l_j + 1$
 end for
 # Exclude finished batches.
 $\mathcal{S}_i \leftarrow \{T_j | T_j \in \mathcal{S}_{i-1}, l_j < l_{\max}\}$
end while

overheads.

4.1. The Cross-Layer Perspective

As discussed in Section 3, expert redundancy within a single MoE layer can only be achieved by expert replication, which inevitably incurs additional memory consumption. However, from a cross-layer perspective, we observe that redundancy inherently exists in the inference system: experts of other layers can be naturally considered as “redundant” for the current layer. Since these experts are already resident in GPU memory and ready for immediate usage without introducing any overhead, this observation suggests a promising solution to the current limitations.

Such potential has been overlooked previously, mainly due to the constraint of layer-wise sequential execution order of models, which prevents experts of other layers from directly participating in the computation. However, we observe that there are no dependencies among multiple micro-batches, making it feasible for micro-batches to simultaneously reside at different MoE layers. As illustrated in Figure 1, the current layers of 4 micro-batches (l_j for $j \in \{0, 1, 2, 3\}$) can differ from each other, while the inference process remains lossless as long as each micro-batch follows the sequential order of itself (each l_j increments by 1 after one MoE computation of micro-batch j). The workloads of batches can be jointly executed (by multiple launches of

their kernels) in one step, regardless of whether they are on the same layer.

Interestingly, cross-layer workload combination is worst-case performance-safe and highly likely to create load-balancing opportunities.

Worst-case safe. According to Equation (2), the effective workload of combining multiple workloads w_j is $\max(\sum_j w_j)$, whereas executing them separately would result in $\sum_j \max(w_j)$. Since we always have $\max(\sum_j w_j) \leq \sum_j \max(w_j)$, the combination never leads to degradation.

Mostly effective. The worst case (when equality is achieved) only occurs when the workloads peak at the same device (i.e., $\operatorname{argmax}_d(w_j)$ is identical for all j), while its probability decreases when the number of devices grows (workloads are increasingly likely to peak on different devices with a large D). In other cases, the combination can reduce the effective workload due to strict inequality that $\max(\sum_j w_j) < \sum_j \max(w_j)$, accelerating MoE computation. More specifically, the imbalance of each workload w_j is “compensated” by those w_k with $\operatorname{argmax}_d(w_k) \neq \operatorname{argmax}_d(w_j)$.

The empirical statistics of worst-case occurrences are reported in Table 2, and the effectiveness of cross-layer workload combination is further demonstrated in Figure 6 and Figure 9.

4.2. Scheduling

Based on cross-layer expert redundancy and workload combination, EasyBalance mitigates load imbalance through micro-batch scheduling. The end-to-end procedure is demonstrated in Algorithm 1.

At each MoE-computation step, the scheduler selects a subset of micro-batches (Runset_i) from the pool ($\mathcal{S}_{i-1}$) and executes the combined workload, while unselected batches are deferred and remain in $\mathcal{S}_i$ for future scheduling. Among the scheduled batches, those that have not reached the final layer (i.e., $l_j < l_{\max}$) will advance to the next layer, progressing through attention, routing and all-to-all dispatching, and become ready for the next scheduling step. This scheduling mechanism allows micro-batches to reside on different layers during the procedure, and cross-layer imbalance mitigation is leveraged. The iterative process continues until all batches complete their execution at the last layer.

As the workload distributions of future layers are unknown at the current layer, the scheduling algorithm must be greedy. We explore several heuristic strategies, and discover that selecting the subset that maximizes GPU utilization generally yields the best performance (see Section 5.2). Additionally, to avoid executing overly small subsets—where computa-

tion and communication overlap becomes ineffective—we introduce a minimum threshold m on the size of RunSet: the scheduling algorithm is required to return a new Runset with $|\text{Runset}| \geq m$. If $|\mathcal{S}| < m$, all available workloads will be executed regardless of imbalance. We suggest the value of m to be $0.5 \sim 0.75 \times$ of the number of micro-batches, according to empirical evidences in Section 5.4.1.

EasyBalance provides instant flexibility across tasks, as scheduling decisions are made solely based on current workload distributions and do not rely on task-specific routing statistics. Moreover, it incurs negligible additional overhead, as no expert replication or migration is involved, addressing the aforementioned limitation on scalability. The scheduling algorithm operates only on small-sized routing metadata, and its runtime cost is lightweight and negligible compared to the end-to-end inference latency (see empirical results in Table 3).

5. Experiments

Models. We evaluate EasyBalance on 3 representative open-source MoE models: Qwen3-30B-A3B-Instruct-2507 (Qwen3-30B) (Yang et al., 2025), Moonlight-16B-A3B-Instruct (Moonlight-16B) (Liu et al., 2025a) and Qwen3-235B-A22B-Instruct-2507 (Qwen3-235B). Except for results in Section 5.3, we use the standard expert-device mapping where experts are evenly distributed across devices. The number of activated experts per token follows the default value in model configurations.

Benchmark. We use LongBench (Bai et al., 2024) as the evaluation benchmark. LongBench is designed for large-scale and long-context inference, covering a diverse set of tasks including passage comprehension, question answering, information retrieval, summarization, and code generation. This task diversity induces highly heterogeneous routing patterns across inputs, making LongBench well suited for evaluating the robust flexibility of methods.

Configurations. All experiments are conducted on a single node equipped with $8 \times \text{A800-SXM4}$ GPUs 80GB, interconnected via NVLink. Unless otherwise stated, the expert parallelism size is set to 8. The batch sizes and token sequence lengths of each GPU are set to $16 \times 4\text{K}$ and 12×512 for smaller (Qwen3-30B and Moonlight-16B) and larger (Qwen3-235B) models respectively. The number of micro-batch splits is 4 with m set to 3, as this setting typically achieves optimal performances.

Metrics. We evaluate inference speed with end-to-end latency, which directly reflects the empirical performances. Additionally, we report effective workload (defined in Equation (2)) and GPU utilization (defined in Equation (3)) as quantitative indicators of load imbalance. For readability, we report GPU “under-utilization” as $1 - u$. The latencies

	MaxUtil	CumUtil	DiffPeak
Scheduling(ms)	0.277	0.267	0.130
End-to-end(s)	8.21	8.27	8.24

Table 3. Scheduling latency(ms) per step and end-to-end latency(s) with different strategies upon Qwen3-30B and 2wikimqa. The batch size and sequence length are 128 and 4K, and the number of micro-batches is 4.

are averaged over 8 runs to minimize the impact of system fluctuations.

5.1. Main Results

Figure 2 presents the end-to-end latencies and GPU under-utilization of tested models across various tasks. As shown, EasyBalance consistently improves inference-speed performance across models and task categories, significantly alleviating GPU under-utilization caused by expert load imbalance by mostly more than 40% (≥ 0.35 to ≈ 0.2). The consistent effectiveness across tasks demonstrates strong robustness and flexibility of our method, which is attributed to the design that does not require expert-mapping modifications.

5.2. Scheduling Algorithms

The scheduling procedure incurs only negligible overhead, as shown in Table 3. However, while the utilization-maximization (**MaxUtil**) scheduling strategy generally achieves best performances, its overhead grows exponentially with the number of batches ($O(2^n)$ with batch size n). Therefore, we additionally evaluate two alternative heuristic algorithms whose overhead scales linearly ($O(n)$).

Denote the workload distribution of micro-batch i across D devices as $\mathbf{w}_i = (w_i^0, w_i^1, \dots, w_i^{D-1})$, the algorithms are:

Cumulated Utilization (CumUtil). A batch will be scheduled only if its inclusion improves the overall GPU utilization. The algorithm evaluates batches sequentially from 0 to $N - 1$, with the first batch always scheduled. Formally, at step i , given the currently chosen workload set $S_{i,j}$, batch $j + 1$ is added to form $S_{i,j+1}$ if

$$U(S_{i,j} \cup \{\mathbf{w}_{i,j+1}\}) > U(S_{i,j}),$$

where $U(\cdot)$ denotes GPU utilization. Otherwise $S_{i,j+1} = S_{i,j}$. $S_{i,0} = \{0\}$.

Different Peak Devices (DiffPeak). A batch will be scheduled only when the peak device of its workload differs from those of all chosen micro-batches. Formally, at step i , batch $j + 1$ is scheduled if

$$\operatorname{argmax}_d(\mathbf{w}_{i,j+1}) \notin \{\operatorname{argmax}_d(\mathbf{w}_{i,k}) | \mathbf{w}_{i,k} \in S_{i,j}\}.$$

Task	Method	Expert-Parallel Size		
		2	4	8
Qwen3-30B				
2wikimqa	w/o EB	0.10	0.21	0.37
	w/ EB	0.04	0.12	0.21
repobench-p	w/o EB	0.10	0.24	0.38
	w/ EB	0.04	0.12	0.23
trec	w/o EB	0.08	0.20	0.35
	w/ EB	0.04	0.10	0.21
Moonlight-16B				
2wikimqa	w/o EB	0.06	0.18	0.33
	w/ EB	0.03	0.11	0.21
repobench-p	w/o EB	0.12	0.29	0.46
	w/ EB	0.05	0.17	0.29
trec	w/o EB	0.07	0.21	0.38
	w/ EB	0.04	0.13	0.26

Table 4. GPU under-utilization of Qwen3-30B and Moonlight-16B on representative tasks. The batch size for EP=2,4,8 is 32,64,128 respectively. EB stands for EasyBalance.

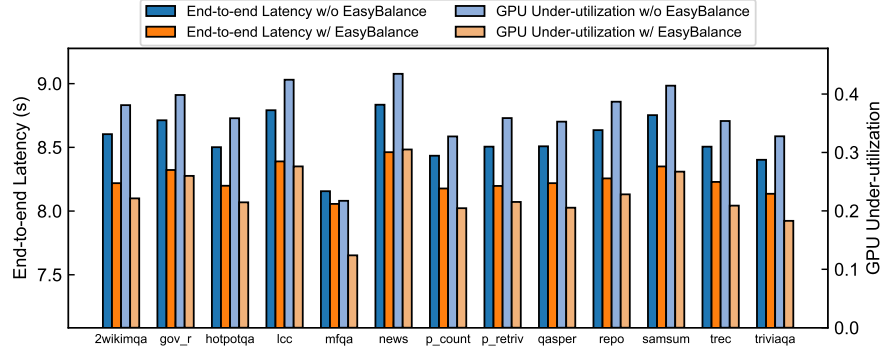
Compared to the original algorithm, these two alternatives do not account for the interaction between workloads across micro-batches, but only each independent workload. The results in Figure 3 show that, while these alternatives underperform the MaxUtil strategy, they both outperform the vanilla baseline (i.e., no cross-layer combination), suggesting that the robust effectiveness of EasyBalance does not critically depend on a specific scheduling heuristic.

5.3. Orthogonality to EPLB

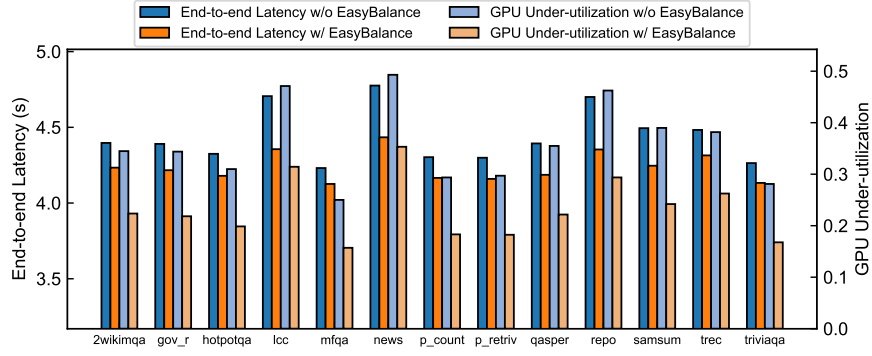
EasyBalance is orthogonal to load balancing methods that require expert-map modification, as it is agnostic to the mapping itself. To validate this, we evaluate EasyBalance in combination with EPLB (DeepSeek AI, 2025). We configure EPLB’s expert placements using the routing history from all 13 representative tasks. The results in Figure 5 show that EasyBalance consistently provides additive performance gains, confirming the orthogonality.

Figure 6 shows the maximum and effective workloads of each scheduling step, where the two metrics are defined as $\sum_j \max(\mathbf{w}_{i,j})$ and $\max \sum_j \mathbf{w}_{i,j}$ respectively. The results show that EasyBalance effectively reduces effective workloads in most cases through cross-layer balancing opportunities ($\sum_j \max(\mathbf{w}_{j,i}) < \max \sum_j \mathbf{w}_{j,i}$).

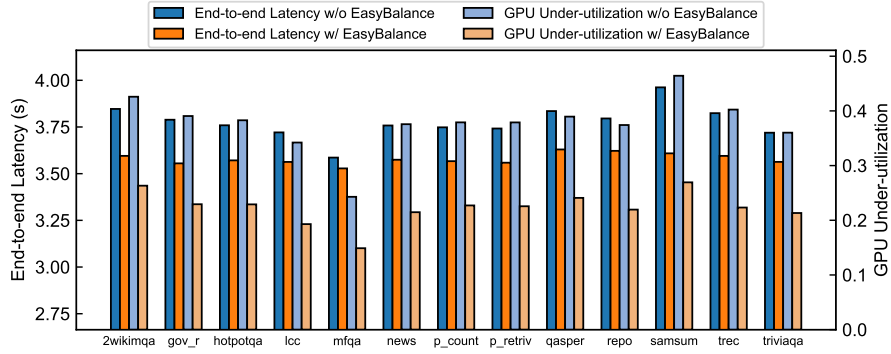
Notably, the performance gains of EPLB differ substantially across task types, highlighting the task-dependent nature of its gains and therefore the limited flexibility of existing map-



(a) Qwen3-30B



(b) Moonlight-16B



(c) Qwen3-235B

Figure 2. End-to-end latency(s) and GPU under-utilization across tasks and models. Lower is better.

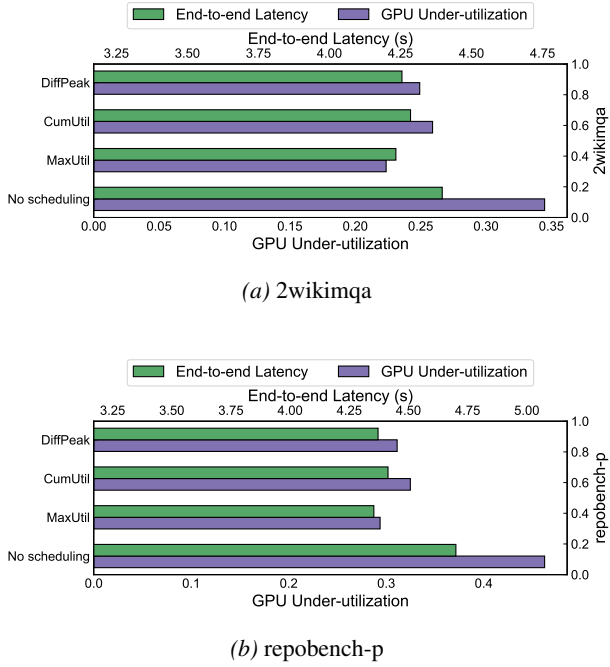


Figure 3. End-to-end latency(s) and GPU under-utilization of different scheduling algorithms, applied to Moonlight-16B on representative tasks. Lower is better.

modification methods. In contrast, EasyBalance achieves consistent and stable effectiveness across all evaluated tasks.

5.4. Ablations

The ablation studies are about configurations of micro-batching and expert parallelism.

5.4.1. MICRO-BATCHING

We vary both the number of micro-batches and the minimum execution threshold m . The results in Figure 4 demonstrate that the effectiveness of EasyBalance remains consistent across different configurations. The value of m plays a critical role in the scheduling behavior: a smaller m may restrict the scheduler to explore more balancing opportunities, while a larger one may force imbalanced workloads to be scheduled. Based on the empirical results, we suggest the value of m to be $0.5 \sim 0.75 \times$ of the number of micro-batches.

Choosing an appropriate micro-batch size is essential for achieving optimal inference performance. Excessively small sizes (e.g., 2) result in insufficient imbalance mitigation, whereas overly large sizes reduce computation intensity and cause slowdown (Leviathan et al., 2023). Notably, the optimal performance is achieved with 4 micro-batches, while the highest GPU utilization occurs at a size of 8. This suggests the potential of even greater performance gains that

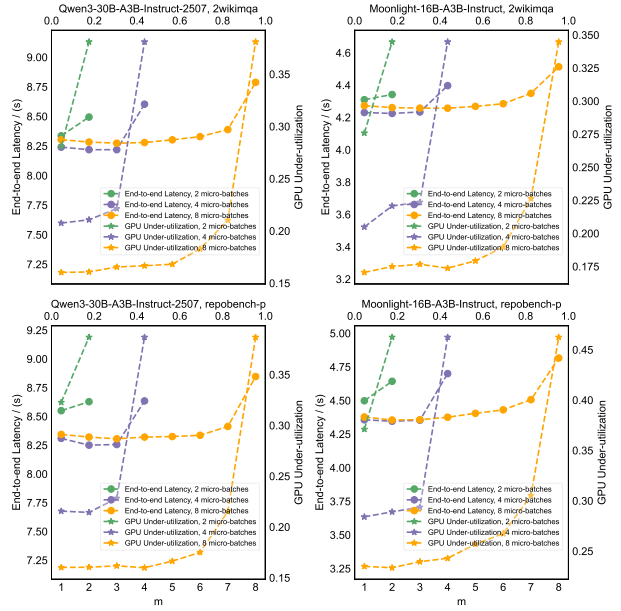


Figure 4. End-to-end latency(s) and GPU under-utilization of different micro-batches and m on representative tasks. Results are the best among scheduling algorithms in Section 5.2. Lower is better.

can be realized under heavier or more imbalanced inference workloads.

5.4.2. PARALLELISM CONFIGURATIONS

Table 4 reports the GPU under-utilization of various expert-parallel (EP) configurations. To ensure a fair comparison, we proportionally adjust the global batch size so that the per-device capacity remains constant (e.g., 32, 64, 128 for EP size = 2, 4, 8 respectively). The results indicate that EasyBalance consistently reduces the effective workloads across all EP configurations.

As demonstrated, larger EP sizes generally result in lower GPU utilization, which is consistent with prior studies (OpenLM.ai, 2025; DeepSeek AI, 2025). This is because co-located experts can absorb each other’s workload imbalance within a device. As the EP size increases, each device hosts fewer experts, reducing local absorbing opportunities before routing skew translates into inter-device imbalance. Therefore, the limitation on scalability is crucial for load-balancing methods, while EasyBalance is designed to address this challenge.

6. Related Works

Distributed MoE Inference. Existing works focus on various aspects of distributed MoE inference acceleration. Sc-MoE (Cai et al., 2025) adopts a shortcut connection to break layerwise dependencies between computation and commu-

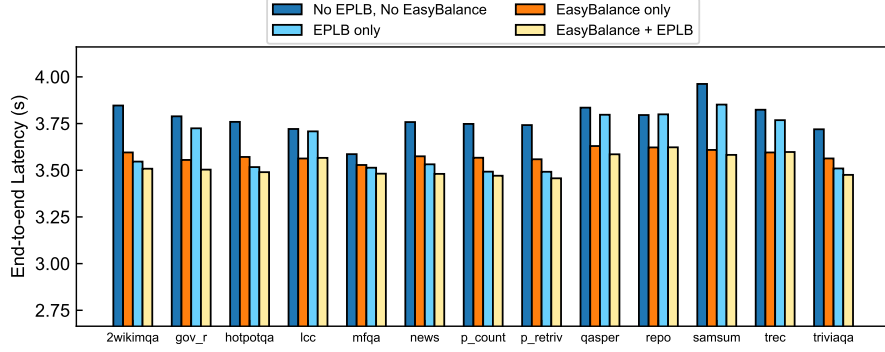


Figure 5. End-to-end latency(s) and GPU under-utilization with/without EPLB on Qwen3-235B. The EPLB expert placement is generated from the routing distribution of all 13 representative tasks of LongBench, with 128 physical experts.

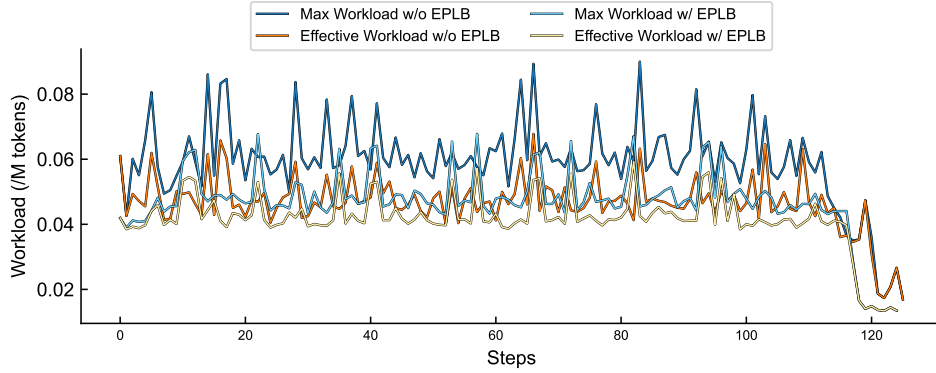


Figure 6. Maximum and effective workloads(/M tokens) of Qwen3-235B on multi_news in one run. The EPLB expert placement is generated from the routing distributions of all 13 representative tasks of LongBench, with 128 physical experts.

nication. Tutel (Hwang et al., 2023) designs an identical distribution layout of MoE models for switchable parallelism and dynamic pipelining. KTransformers (Chen et al., 2025) develops faster CPU kernels and proposes expert deferral for device-heterogeneous distributed MoE inference. DeepSpeed-MoE (Rajbhandari et al., 2022) implements hierarchical all-to-all based on tensor- and expert-parallel topologies to reduce communication overheads. Occult (Luo et al., 2025) combines co-activated experts onto the same device for efficient all-to-all communication.

Expert-Parallel Load Balancing. Existing works on EP load balancing mainly address the problem through routing statistic analysis, expert replication, and expert migration. Lina (Li et al., 2023) observes that tokens routed to the same expert in one layer are more likely to be routed to another same expert in the following layer, and exploits this property to profile routing patterns and guide expert replication. EPLB (DeepSeek AI, 2025) introduces expert redundancy by creating replicated experts and dynamically adjusting expert placement based on historical routing information. Harmony (Doucet et al., 2025) further adapts expert

workload allocation according to current routing results, while migrating experts in the background of computation to reduce the impact of additional overhead.

Existing load-balancing strategies are largely constrained in flexibility and scalability, due to their reliance on modifying expert-device mappings. EasyBalance addresses this limitation by exploring an orthogonal direction: expert-map-independent load balancing, which has been largely unexplored in prior studies.

7. Conclusion

This paper proposes EasyBalance, a cross-layer load-balancing strategy for expert-parallel MoE inference. The key insights are cross-layer expert redundancy and workload combination. Without requiring expert replication or migration, EasyBalance creates load-balancing opportunities by scheduling micro-batches to execute MoE computation across different layers. EasyBalance consistently accelerates distributed MoE inference, improving GPU utilization by mostly over 40%.

Acknowledgments

This work is partially supported by the NSF of China (under Grant 92364202), and Major Program of ISCAS (Grant No. ISCAS-ZD-202402).

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., et al. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pp. 3119–3137, 2024.
- Cai, W., Jiang, J., Qin, L., Kim, S., Huang, J., et al. Shortcut-connected expert parallelism for accelerating mixture of experts. In *Forty-second International Conference on Machine Learning*, 2025.
- Chen, H., Xie, W., Zhang, B., Tang, J., Wang, J., Dong, J., Chen, S., Yuan, Z., Lin, C., Qiu, C., et al. Ktransformers: Unleashing the full potential of cpu/gpu hybrid inference for moe models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pp. 1014–1029, 2025.
- DeepSeek AI. Eplb: Expert parallelism load balancer. <https://github.com/deepseek-ai/EPLB>, 2025.
- Doucet, Z., Sharma, R., de Vos, M., Pires, R., Kermarrec, A.-M., and Balmau, O. Harmony: Efficient multi-gpu inference of moe models. *arXiv preprint arXiv:2506.12417*, 2025.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, pp. 30016–30030, 2022.
- Huang, H., Ardalani, N., Sun, A., Ke, L., Lee, H.-H. S., Bhosale, S., Wu, C.-J., and Lee, B. Toward efficient inference for mixture of experts. *Advances in Neural Information Processing Systems*, 37:84033–84059, 2024.
- Hwang, C., Cui, W., Xiong, Y., Yang, Z., Liu, Z., Hu, H., Wang, Z., Salas, R., Jose, J., Ram, P., et al. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems*, 5:269–287, 2023.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.
- Li, J., Jiang, Y., Zhu, Y., Wang, C., and Xu, H. Accelerating distributed {MoE} training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pp. 945–959, 2023.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Liu, J., Su, J., Yao, X., Jiang, Z., Lai, G., Du, Y., Qin, Y., Xu, W., Lu, E., Yan, J., et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025a.
- Liu, X., Wang, Y., Fu, F., Miao, X., Zhu, S., Nie, X., and Cui, B. Netmoe: Accelerating moe training through dynamic sample placement. In *The Thirteenth International Conference on Learning Representations*, 2025b.
- Luo, S., Li, P., Peng, J., Zhao, Y., Cao, Y., Cheng, Y., and Chen, T. Occult: Optimizing collaborative communications across experts for accelerated parallel moe training and inference. In *Forty-second International Conference on Machine Learning*, 2025.
- OpenLM.ai. Large-scale expert parallelism. <https://openlm.ai/large-scale-expert-parallelism>, 2025.

- Rajbhandari, S., Li, C., Yao, Z., Zhang, M., Aminabadi, R. Y., Awan, A. A., Rasley, J., and He, Y. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*, pp. 18332–18346. PMLR, 2022.
- SGLang Team. Sglang: Efficient execution of structured generation programs. <https://github.com/sgl-project/sglang>, 2024.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Wang, B., Wang, B., Wan, C., Huang, G., Hu, H., Jia, H., Nie, H., Li, M., Chen, N., Chen, S., et al. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding. *arXiv preprint arXiv:2507.19427*, 2025.
- Yan, Z., Li, D., Chen, L., Xiong, D., Gao, K., Zhang, Y., Yan, R., Zhang, M., Zhang, B., Jiang, Z., et al. From atop to zcube: Automated topology optimization pipeline and a highly cost-effective network topology for large model training. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pp. 861–881, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

A. Workload Distributions across Tasks

Figure 7 demonstrates the routing pattern of each layer of Qwen3-30B models, with lighter colors indicating lower workloads. According to the results, the routing distributions are highly task-dependent, posing limitations upon flexibility of load balancing methods that require expert-map modification.

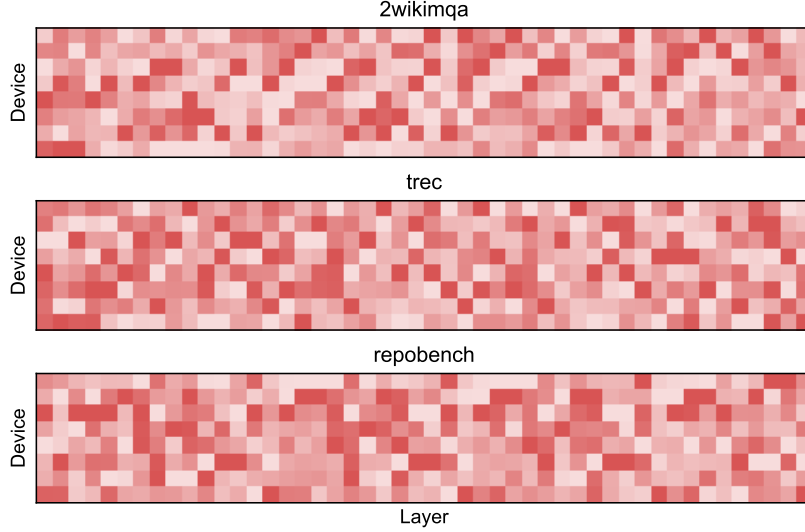


Figure 7. Workload distributions of Qwen3-30B on representative tasks under EP=8. Lighter colors indicate lower workloads.

B. Clarification of Figures

We report end-to-end latencies as the performance metric, yet EasyBalance targets only at MoE computation, rather than attention. Formally, we have

$$t_{e2e} = t_{expert} + t_{attention}$$

Note that under the same settings of batch size and sequence length, $t_{attention}$ is constant across different tasks (irrelevant to MoE imbalance). Therefore, we set the start of y-axis of figures to be $t_{attention}$ for better demonstration of “expert latency”.

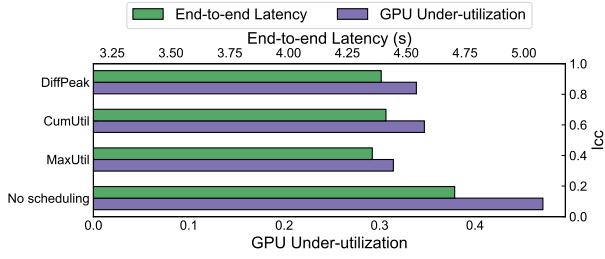
C. More Results

C.1. EPLB with Redundant Experts

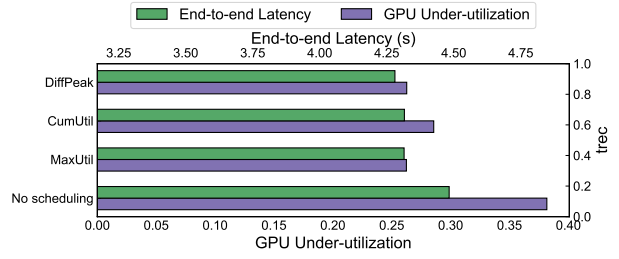
EPLB supports redundant expert placement, where redundant experts are replications of some hot experts from other EP ranks. However, the results in Table 5 show that introducing redundant experts yields only marginal improvements on load balancing, while incurring substantial memory overhead. We evaluate Qwen3-30B with 128 logical experts under EP=8 and observe that the utilization improvement for 136 physical experts (8 replicas, 1 for each EP rank) is marginal, especially compared to the significant effectiveness of EasyBalance. The results also show that EasyBalance is orthogonal to expert replication, while incurring no memory overhead.

Task	EPLB(128)	EPLB(128)+EasyBalance	EPLB(136)	EPLB(136)+EasyBalance
2wikimqa	0.23	0.14	0.22	0.12
trec	0.29	0.16	0.28	0.17

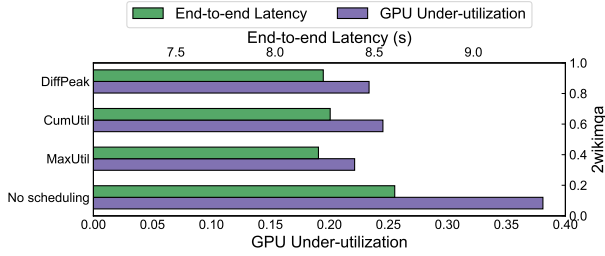
Table 5. GPU under-utilization of EPLB with various numbers of physical experts on Qwen3-30B. The batch size and sequence length are 128 and 4K, respectively.



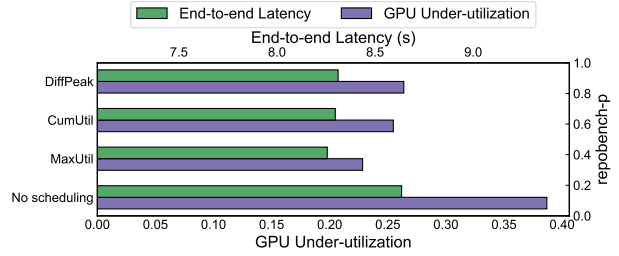
(a) Moonlight-16B, lcc



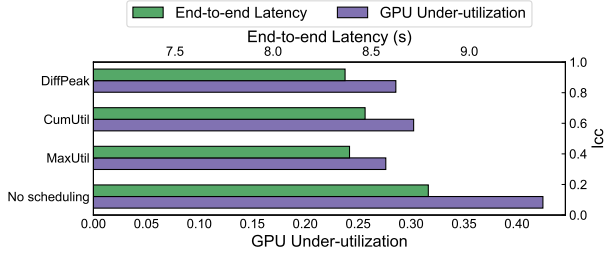
(b) Moonlight-16B, trec



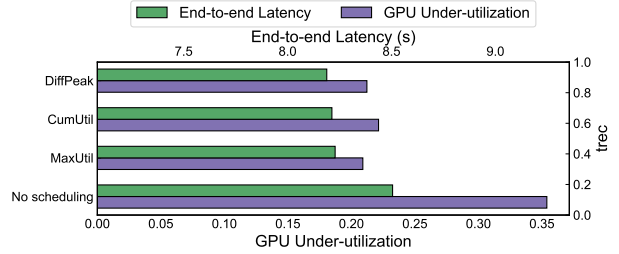
(c) Qwen3-30B, 2wikimqa



(d) Qwen3-30B, repobench-p



(e) Qwen3-30B, lcc



(f) Qwen3-30B, trec

Figure 8. End-to-end latency(s) and GPU under-utilization of different scheduling algorithms across tasks and models.

C.2. Scheduling Algorithms

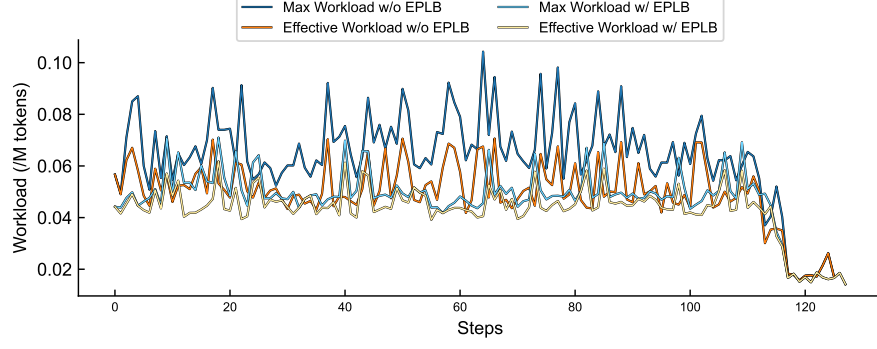
Figure 8 shows more results about different scheduling algorithms. The results indicate that MaxUtil may underperform other strategies in some cases, yet all the algorithms effectively reduce end-to-end latency and GPU under-utilization.

C.3. Effectiveness of Cross-Layer Workload Combination

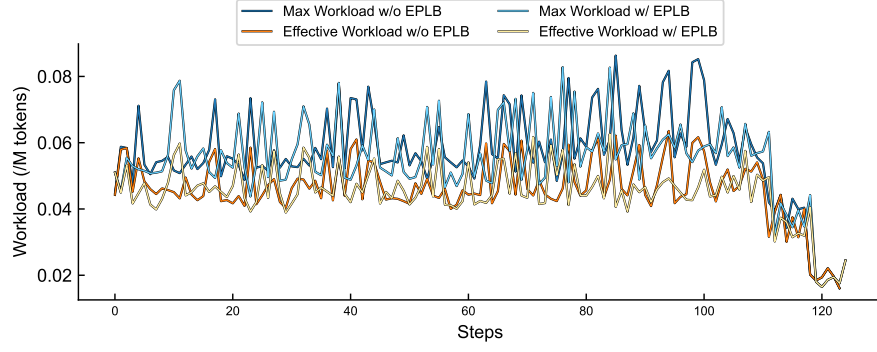
Figure 9 shows additional results of per-step workload statistics upon Qwen3-235B across more tasks, as a complementary to Figure 6. Results show that workloads can mitigate each other’s imbalance in most cases, demonstrating the effectiveness of our method.

D. Limitations

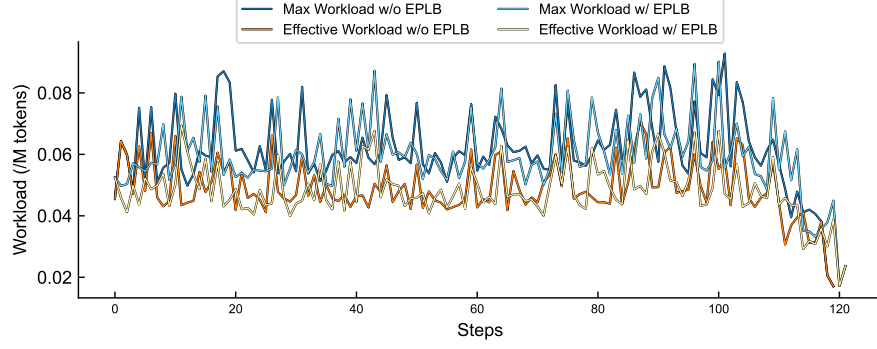
EasyBalance is effective under distributed MoE inference, while it has no acceleration on non-MoE models or no-EP scenarios. EasyBalance currently requires micro-batching, while the potential of applying it to a single sequence (by splitting the sequence into pieces and run them on different layers) remain unexplored. The experiments are conducted only on A800-SXM4 80GB GPUs, leaving its empirical effectiveness on other platforms uncertain. EasyBalance is potentially effective under Attention-FFN Disaggregation (Wang et al., 2025), while the effectiveness remains unverified.



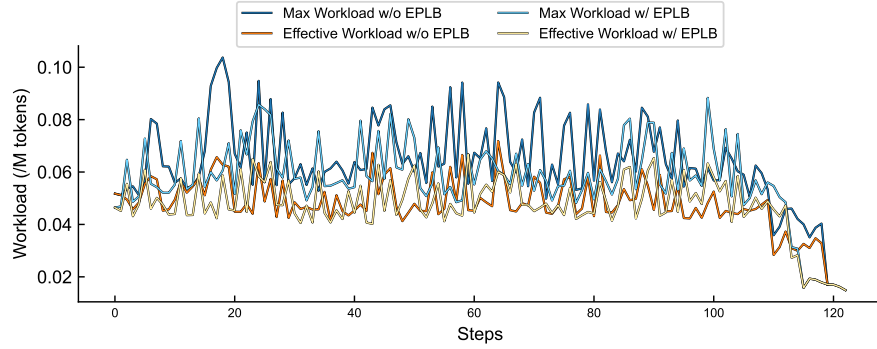
(a) 2wikimqa



(b) lcc



(c) repobench-p



(d) trec

Figure 9. Maximum and effective workloads of Qwen3-235B across tasks in one run. The EPLB expert placement is the same as Figure 6.